pos-elera-sales-compliance — YAMLess AKS Run 2026-11-11

pos-elera-sales-compliance — YAMLess AKS Run

krogertechnology/pos-elera-sales-compliance | Target: AKS | 2026-11-11

Run Summary

Field	Value
Repo	`krogertechnology/pos-elera-sales-compliance`
Target	Azure Kubernetes Service (AKS)
YAMLess branch	develop @ a758234
Issues applied	#26–#36 (all fixes through Cosmos DB, secret denylist, warning threshold)
Date	2026-11-11
Output	38 .tf files, 8 .yaml files

Detected Profile

Field	Value	Confidence
Runtime	Java (JDK 25)	high
Shape	http-service	high
Port	8080	—
Health check path	`/actuator/health`	(from spring-boot-starter-actuator)
Memory limit	1152Mi	(from helm/values.yaml)
Memory request	256Mi	(from helm/values.yaml)
Database	MongoDB → Cosmos DB	high
Cache	Redis → Azure Cache for Redis	high
Queue	RabbitMQ → Azure Service Bus	high
Min replicas	1	(no replicaCount in this repo's Helm)
Max replicas	10	—
Secrets	39	high

Terraform Modules Generated

```
terraform/
├── main.tf
```

```
│   ├── module "vnet"
│   ├── module "aks_cluster"
│   ├── module "keyvault"
│   ├── module "aks_workload_identity"
│   ├── module "cosmos"          ← Cosmos DB for MongoDB (autoscale 10,000 RU)
│   ├── module "redis"           ← Azure Cache for Redis (Premium P1)
│   └── module "servicebus"      ← Azure Service Bus (Standard)
├── variables.tf
├── outputs.tf
├── governance.tf
├── backend.tf
└── modules/
    ├── cosmos/        (azurerm_cosmosdb_account — MongoDB kind, writes mongodb-url to KV)
    ├── redis/
    ├── servicebus/
    ├── keyvault/
    ├── vnet/
    ├── aks_cluster/
    └── aks_workload_identity/
```

Kubernetes Manifests Generated

```
k8s/
├── namespace.yaml
├── service-account.yaml
├── secret-provider-class.yaml   ← 39 KV secret refs
├── deployment.yaml              ← probes: /actuator/health, limits: 1152Mi
├── service.yaml
├── hpa.yaml
├── network-policy.yaml
└── ingress.yaml
```

Cosmos DB Module Config

Driven by `golden-profiles/azure-cosmos-db.json` for `production` environment:

```
module "cosmos" {
  source            = "./modules/cosmos"
  consistency_level = "BoundedStaleness"
  throughput_mode   = "autoscale"
  max_autoscale_ru  = 10000
  server_version    = "4.2"
}
```

Secrets (39)

Secret denylist (#34) removed `GF_SECURITY_ADMIN_PASSWORD`, `SWAGGER_URL`, `SWAGGER_CONFIG_URL` that appeared in v2. `MONGODB_URI` and `SPRING_DATA_MONGODB_URI` are marked `infra_managed` (#35) — `mongodb-url` from the Cosmos module is authoritative.

Remaining 39 KV secrets are legitimate application credentials:

Category	Examples
PingFederate / OAuth	`PINGFED_CLIENT_SECRET`, `PING_AUTH_ISSUER_URI`, `PING_AUTH_JWK_SET_URI`, `PING_FED_*` (×8)

Category	Examples
RabbitMQ	`AMQP_BROKER_PASSWORD`, `MQ_PASSWORD`, `SPRING_RABBITMQ_PASSWORD`
External services	`KOUNT_API_KEY`, `KOUNT_BASE_URL`, `ELERA_*` (×3), `COMPLIANCE_BASE_URL`
Spring Cloud Config	`CLOUD_CONFIG_URL`
Redis	`REDIS_HOST`, `REDIS_KEY`
RSA / crypto	`CIS_RSA_PUBLIC_KEY`
Misc endpoints	`DB_URI`, `LOADER_CATALOG_BASE_URL`, `EXTERNAL_SERVICES_*`, etc.

> ⚠️ **Warning:** High secret count (39). Review whether all env vars need to be in Key Vault — consider grouping with a secrets manager pattern or filtering observability vars.

Cost Estimate

Environment	Monthly	Annual
Dev	$170 – $245	$2,040 – $2,940
Staging	$392 – $542	$4,704 – $6,504
Production	**$1,217 – $1,867**	**$14,604 – $22,404**

Production breakdown:

Resource	Cost/mo	Notes
AKS Cluster	$600	Control plane free; system node pool Standard_D2s_v3
Cosmos DB (MongoDB)	$200–$800	Autoscale 2,000–10,000 RU, BoundedStaleness
Redis Cache	$370	Premium P1 (6 GB)
Service Bus	$32–$82	Standard tier
Key Vault	$15	Premium SKU

Open Gaps (all absorbed into #37)

Issue	Gap	Impact
#30	AMQP 0-9-1 → AMQP 1.0 migration risk not surfaced	Silent — app may break on direct reply-to patterns
#32	JMX / management secondary port not detected	Missing containerPort; no NetworkPolicy rule
#33	Spring Cloud Config startup dependency not modeled	Pod will fail to start if config server unreachable; no init container or warning

These are non-blocking for pod scheduling but represent real migration risks. All three are driving use cases for the Runtime Enricher and Migration Risk Evaluator in #37.

YAMLess — https://github.com/zafar-ahmed-kr/YAMLess Date: 2026-11-11